

Resumo do Código Implementado

Este notebook implementa um solucionador didático para o jogo Termo usando uma Árvore Trie, seguindo o enunciado [PAA_TrabalhoPratico3_ArvoreTrie_Alternativo.pdf](#), baseado no jogo do site <https://termooo.com/>. O dicionário é formado por palavras reservadas da linguagem Python com cinco letras, compatíveis com o formato clássico do Termo. A Trie é construída a partir desse dicionário e usada para buscar palavras exatas, listar palavras por prefixo e sugerir possíveis soluções a partir de um chute e das cores retornadas pelo jogo. Verdo, Amarelo e Preto.

A solução também permite acumular respostas anteriores para refinar as sugestões, exibe a Trie e as buscas usando SVG/HTML em células separadas e inclui uma versão offline em que o computador joga sozinho, mostrando as tentativas e o conjunto de possíveis soluções a cada rodada.

Legenda das cores do Termo

- Verde (V) :** a letra existe na palavra secreta e está na posição correta.
- Amarelo (A) :** a letra existe na palavra secreta, mas está em outra posição.
- Preto (P) :** a letra não existe na palavra secreta ou já teve todas as suas ocorrências consideradas por letras verdes/amarelas.

Estrutura das células do notebook

- Resumo e estrutura do notebook.
- Importações, constantes e dicionário de palavras reservadas Python.
- Definição da Árvore Trie.
- Construção da Trie do dicionário.
- Busca exata e busca por prefixo na Trie.
- Visualização SVG/HTML da Trie montada.
- Visualização didática da busca exata.
- Visualização didática da busca por prefixo.
- Cálculo das cores do Termo.
- Filtragem de sugestões usando chute e cores.
- Visualização didática da filtragem por cores.
- Assistente interativo com histórico de respostas.
- Termo offline para o computador jogar sozinho.
- Demonstrações de execução.
- Resultados pedidos no enunciado.

```
In [1]: # Esta célula importa as bibliotecas usadas, define constantes do Termo e monta o dicionário
# com palavras reservadas da Linguagem Python que possuem cinco letras.
# Complexidade para montar o dicionário: O(K * L), em que K é a quantidade de palavras
# reservadas analisadas e L é o tamanho máximo de cada palavra.

from dataclasses import dataclass, field
from html import escape
from IPython.display import HTML, display
import keyword
from collections import Counter

WORD_LENGTH = 5
VERDE = "V"
AMARELO = "A"
PRETO = "P"
CORES_VALIDAS = (VERDE, AMARELO, PRETO)

def normalizar_palavra(palavra):
    """Normaliza palavras para comparação no Termo."""
    return palavra.strip().lower()

PALAVRAS_RESERVADAS_PYTHON = sorted(
    normalizar_palavra(palavra)
    for palavra in keyword.getlist() if getattr(keyword, "softkeyword", [])
    if len(normalizar_palavra(palavra)) == WORD_LENGTH and normalizar_palavra(palavra).isalpha()
)

print(f"Dicionário usado ({len(PALAVRAS_RESERVADAS_PYTHON)} palavras):")
print(PALAVRAS_RESERVADAS_PYTHON)

Dicionário usado (9 palavras):
['async', 'await', 'break', 'class', 'false', 'match', 'raise', 'while', 'yield']
```

```
In [2]: # Esta célula implementa a estrutura de dados Árvore Trie usada para armazenar e recuperar
# as palavras do dicionário.
# Complexidade de inserção: O(L), em que L é o tamanho da palavra inserida.
# Complexidade de busca exata: O(L).
# Complexidade de busca por prefixo: O(P * S), em que P é o tamanho do prefixo e S é a
# quantidade total de caracteres visitados na subárvore de resultados.
# Complexidade para listar todas as palavras: O(N * L), em que N é a quantidade de palavras.

@dataclass
class TrieNode:
    children: dict = field(default_factory=dict)
    is_end: bool = False

class Trie:
    def __init__(self):
        self._root = TrieNode()
        self.size = 0

    def insert(self, word):
        node = self._root
        for char in normalizar_palavra(word):
            node = node.children.setdefault(char, TrieNode())
            if char not in node.children:
                node.is_end = True
                self.size += 1

    def search(self, word):
        node = self._root
        for char in normalizar_palavra(word):
            if char not in node.children:
                return False
            node = node.children[char]
        return node.is_end

    def _find_node(self, prefix):
        node = self._root
        for char in normalizar_palavra(prefix):
            if char not in node.children:
                return None
            node = node.children[char]
        return node

    def starts_with(self, prefix):
        prefix = normalizar_palavra(prefix)
        node = self._find_node(prefix)
        if node is None:
            return []
        return self._collect_words(node, prefix)

    def words(self):
        return self._collect_words(self._root, "")

    def _collect_words(self, node, prefix):
        results = []
        if node.is_end:
            results.append(prefix)
        for char in sorted(node.children):
            results.extend(self._collect_words(node.children[char], prefix + char))
        return results
```

```
In [3]: # Esta célula constrói a Trie a partir do dicionário de palavras reservadas Python.
# Complexidade: O(N * L), em que N é a quantidade de palavras e L é o tamanho das palavras.

def construir_trie(palavras):
    trie = Trie()
    for palavra in palavras:
        trie.insert(palavra)
    return trie

trie_python = construir_trie(PALAVRAS_RESERVADAS_PYTHON)

print(f"Trie construída com {trie_python.size} palavras.")
print(trie_python.words())

Trie construída com 9 palavras.
['async', 'await', 'break', 'class', 'false', 'match', 'raise', 'while', 'yield']
```

```
In [4]: # Esta célula separa as funções de busca usadas pelos algoritmos principais.
# Complexidade de buscar palavra exata: O(L).
# Complexidade de buscar por prefixo: O(P * S), em que P é o tamanho do prefixo e S é a
# quantidade total de caracteres das palavras encontradas.

def buscar_palavra(trie, palavra):
    return trie.search(palavra)

def buscar_por_prefixo(trie, prefixo):
    return trie.starts_with(prefixo)

print("Busca exata 'class':", buscar_palavra(trie_python, "class"))
print("Busca exata 'termo':", buscar_palavra(trie_python, "termo"))
print("Busca por prefixo 'a':", buscar_por_prefixo(trie_python, "a"))

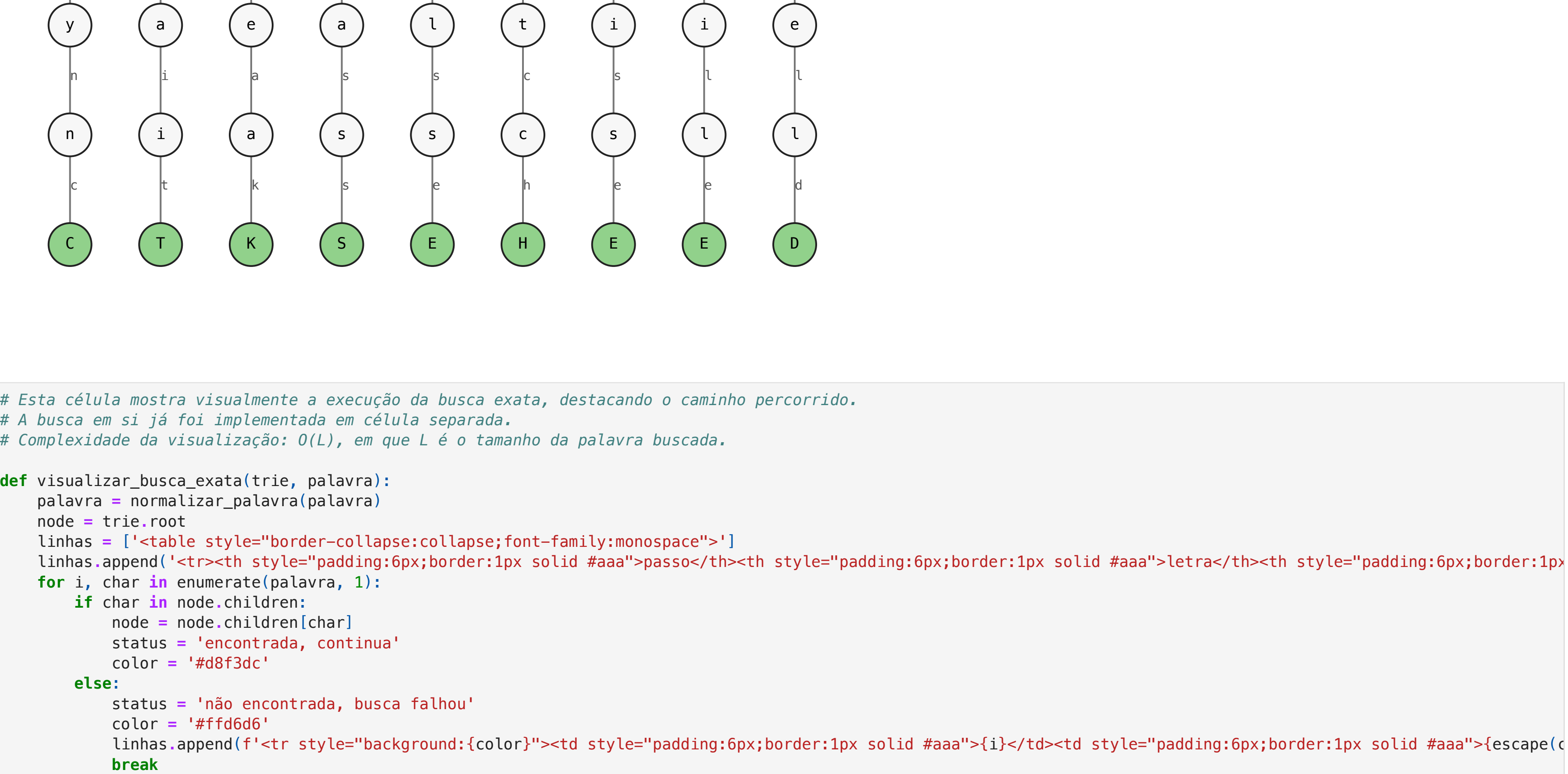
Busca exata 'class': True
Busca exata 'termo': False
Busca por prefixo 'a': ['async', 'await']
```

```
In [5]: # Esta célula exibe a Trie montada em SVG/HTML. Ela é separada da implementação da Trie
# para que os algoritmos possam ser executados sem a visualização.
# Complexidade da visualização: O(M), em que M é o número de nós da Trie.

def coletar_nos_trie(trie):
    nodes = []
    edges = []
    queue = [(trie._root, "raiz", 0, None, "")]
    next_id = 1
    while queue:
        node, label, depth, parent_id, edge_label = queue.pop(0)
        node_id = len(nodes)
        nodes.append({"id": node_id, "label": label, "depth": depth, "end": node.is_end})
        if parent_id is not None:
            edges.append((parent_id, node_id, edge_label))
        for char in sorted(node.children):
            child = node.children[char]
            child_label = char.upper() if child.is_end else char
            queue.append((child, child_label, depth + 1, node_id, char))
            next_id += 1
    return nodes, edges

def svg_trie(trie):
    nodes, edges = coletar_nos_trie(trie)
    levels = {}
    for node in nodes:
        levels.setdefault(node["depth"], []).append(node)
    positions = {}
    width = max(768, max(len(level) for level in levels.values()) * 76)
    height = (max(levels) + 1) * 92 + 40
    for depth, level_nodes in levels.items():
        gap = width / (len(level_nodes) + 1)
        for index, node in enumerate(level_nodes, 1):
            positions[node["id"]] = (gap * index, 40 + depth * 92)
    parts = []
    parts.append(f'<svg width="{width}" height="{height}" viewBox="0 0 {width} {height}" xmlns="http://www.w3.org/2000/svg">')
    parts.append('<style>edge:stroke:#7f7f7f;stroke-width:1.5;node:stroke:#222;stroke-width:1.5;text(font:13px monospace;dominant-baseline:middle;text-anchor:middle);ec')
    for parent, child in edges:
        x1, y1 = positions[parent]
        x2, y2 = positions[child]
        parts.append('<line class="edge" x1="x1" y1="(y1+18)" x2="(x2)" y2="(y2-18)" />')
        parts.append('<text class="edgeText" x="(x1+x2/2)" y="(y1+y2/2)">{escape(label)}</text>')
    for node in nodes:
        x, y = positions[node["id"]]
        fill = "#99d9db" if node["end"] else "#f7f7f7"
        parts.append(f'<rect class="node" cx="x" cy="y" rx="18" fill="{fill}" />')
        parts.append(f'<text class="text" x="x" y="y">{escape(node["label"])}</text>')
        parts.append('</svg>')
    return HTML(''.join(parts))

display(svg_trie(trie_python))
```



```
In [6]: # Esta célula mostra visualmente a execução da busca exata, destacando o caminho percorrido.
# A busca em si já foi implementada em célula separada.
# Complexidade da visualização: O(L), em que L é o tamanho da palavra buscada.

def visualizar_busca_exata(trie, palavra):
    palavra = normalizar_palavra(palavra)
    node = trie._root
    linhas.append('<tr><th style="padding:6px;border:1px solid #aaa">passo</th><th style="padding:6px;border:1px solid #aaa">letra</th><th style="padding:6px;border:1px solid #aaa">status</th></tr>')
    for i, char in enumerate(palavra, 1):
        if char in node.children:
            node = node.children[char]
            status = 'encontrada, continua'
            color = '#d8f3dc'
        else:
            status = 'não encontrada, busca falhou'
            color = '#ffdbdb'
            break
        linhas.append(f'<tr><td style="background:{color}">{i}</td><td style="padding:6px;border:1px solid #aaa">{i}</td><td style="padding:6px;border:1px solid #aaa">{escape(char)}</td></tr>')
    final = 'palavra completa encontrada' if node.is_end else 'prefixo existe, mas não é palavra final'
    color = '#d8f3dc' if node.is_end else '#ffdbdb'
    linhas.append(f'<tr><td style="background:{color}">{len(palavra)}</td><td style="padding:6px;border:1px solid #aaa">{len(palavra)}</td><td style="padding:6px;border:1px solid #aaa">{escape(char)}</td></tr>')
    display(HTML(''.join(linhas)))
```

passo	letra	resultado
1	c	encontrada, continua
2	l	encontrada, continua
3	a	encontrada, continua
4	s	encontrada, continua
5	s	encontrada, continua
fim	-	palavra completa encontrada

passo	letra	resultado
1	t	não encontrada, busca falhou

passo	letra	resultado
1	c	encontrada, continua
2	a	não encontrada, busca falhou

```
In [7]: # Esta célula mostra visualmente a execução da busca por prefixo e as palavras coletadas
# na subárvore encontrada.
# Complexidade da visualização: O(P * S), em que P é o tamanho do prefixo e S é a soma
# dos caracteres coletados nos resultados.

def visualizar_busca_prefixo(trie, prefixo):
    prefixo = normalizar_palavra(prefixo)
    results = buscar_por_prefixo(trie, prefixo)
    chips = ''
    html = f'<div style="font-family:monospace;border:1px solid #aaa;padding:10px;max-width:760px">
    <div>strong>Prefixo pesquisado:</div> {escape(prefixo)}</div>
    <div>strong>Total encontrado:</div> {len(results)}</div>
    <div style="margin-top:8px">{chips if chips else 'nenhuma palavra encontrada-emo'}</div>
    </div>
    ...
    display(HTML(html))

visualizar_busca_prefixo(trie_python, "a")
visualizar_busca_prefixo(trie_python, "u")
```

```
Prefixo pesquisado: a
Total encontrado: 2
[async] [await]

Prefixo pesquisado: u
Total encontrado: 1
[while]
```

```
In [8]: # Esta célula implementa a regra de cores do Termo para comparar um chute com uma palavra
# secreta. A regra é feita repetidas vezes marcando verdes primeiro e amarelas apenas se
# ainda houver ocorrência disponível na palavra secreta.
# Complexidade: O(L), em que L é o tamanho da palavra.

def calcular_cores(termo, secreta, chute):
    secreta = normalizar_palavra(secreta)
    chute = normalizar_palavra(chute)
    if len(secreta) != len(chute):
        raise ValueError("A palavra secreta e o chute devem ter o mesmo tamanho.")
    cores = [PRETO] * len(chute)
    for i, (s, c) in enumerate(zip(secreta, chute)):
        if s == c:
            cores[i] = VERDE
        else:
            restantes[i] += 1
    for i, c in enumerate(chute):
        if restantes[i] == 0:
            continue
        if restantes[i] > 0:
            cores[i] = AMARELO
            restantes[i] -= 1
    return ''.join(cores)

def normalizar_cores(cores):
    cores = ''.join(cores).replace(' ', '').replace('.', '').upper()
    if any(c not in CORES_VALIDAS for c in cores):
        raise ValueError("Use apenas A para Amarelo, V para Verde e P para Preto.")
    len(cores) != WORD_LENGTH:
        raise ValueError(f"Informe exatamente {WORD_LENGTH} cores.")
    return cores

print("Cores para chute 'await' contra secreta 'async':", calcular_cores_termo("async", "await"))
print("Cores para chute 'await' contra secreta 'xaylw':", calcular_cores_termo("xaylw", "await"))
print("Cores para chute 'while' contra secreta 'lwehl':", calcular_cores_termo("lwehl", "while"))
print("Cores para chute 'lwehl' contra secreta 'lwxhl':", calcular_cores_termo("lwxhl", "lwehl"))

Cores para chute 'await' contra secreta 'async': VPPPP
Cores para chute 'await' contra secreta 'xaylw': AAPPP
Cores para chute 'while' contra secreta 'lwehl': VAAAA
Cores para chute 'lwehl' contra secreta 'lwxhl': AAAAP
```

```
In [9]: # Esta célula implementa a filtragem das possíveis soluções usando a Trie e as cores
# retornadas pelo Termo. A comparação é feita simulando as cores de cada candidato e assim,
# letras repetidas são descartadas corretamente.
# Complexidade para um chute: O(N * L), em que N é o número de palavras da Trie e L é o
# tamanho das palavras.
# Complexidade com H respostas anteriores: O(H * N * L).

def palavra_compativel(candidata, chute, cores):
    cores = normalizar_cores(cores)
    return calcular_cores_termo(candidata, chute) == cores

def sugerir_palavras(trie, chute, cores, historico=None):
    chute = normalizar_palavra(chute)
    cores = normalizar_cores(cores)
    if len(chute) != WORD_LENGTH:
        raise ValueError("O chute deve ter {WORD_LENGTH} letras.")
    restricoes = list(historico or []) + [(chute, cores)]
    sugestoes = []
    for candidata in trie.words():
        if all(palavra_compativel(candidata, chute, antigo, cores_antigos) for chute_antigo, cores_antigos in restricoes):
            sugestoes.append(candidata)
    return sugestoes

print("Sugestões para chute 'await' com resposta 'VPPPP':")
print(sugerir_palavras(trie_python, "await", "VPPPP"))
```

```
Sugestões para chute 'await' com resposta 'VPPPP':
['async']
```

```
In [10]: # Esta célula mostra visualmente a filtragem por cores. Ela fica separada da função
# sugerir_palavras para manter a lógica principal independente da visualização.
# Complexidade da visualização: O(H * L), igual à filtragem, mais o custo de montar HTML.

COR_HTML = (VERDE: "#6aa664", AMARELO: "#c9b458", PRETO: "#787c7e")

def pintar_palavra(palavra, cores):
    blocos = []
    for letra, cor in zip(palavra.upper(), normalizar_cores(cores)):
        blocos.append(f'<span style="display:inline-block;width:34px;height:34px;line-height:34px;text-align:center;margin:2px;color:white;background:{COR_HTML[cor]}>{letra}</span>')
    return ''.join(blocos)

def visualizar_filtragem(trie, chute, cores, historico=None):
    sugestoes = sugerir_palavras(trie, chute, cores, historico)
    itens = ''
    html = f'<div style="font-family:monospace;border:1px solid #aaa;padding:12px;max-width:760px">
    <div>strong>Chute:</div> {pintar_palavra(chute, cores)}</div>
    <div style="margin-top:8px">strong>Sugestões {len(sugestoes)}</div>
    <div>{itens if itens else 'nenhuma candidata compativel'}</div>
    </div>
    ...
    display(HTML(html))

visualizar_filtragem(trie_python, "await", "VPPPP")
```

```
Chute: A W A A T T
Sugestões (1):
1. async
```

```
In [11]: # Esta célula implementa uma rotina interativa para o usuário informar chutes e cores
# retornadas pelo site https://termooo.com. As respostas anteriores são acumuladas para
# melhorar as sugestões.
# Complexidade por rodada: O(H * N * L), em que H é a quantidade de respostas acumuladas.

# Chute #1: async
# Resposta (ex.: AVPPP): AAAPA
# Chute #2: class
# Resposta (ex.: AVPPP): VVVVV

def termo_solver_interativo():
    historico = []
    rodada = 1
    comandos_saida = ["sair", "esc", "fim", "q"]
    print("Informe cores com A=Amarelo, V=Verde, P=Preto. Digite 'sair' para encerrar.")
    try:
        while True:
            chute = input(f"Chute #{rodada}: ").strip()
            if chute.lower() in comandos_saida:
                break
            cores = input("Resposta (ex.: AVPPP: ").strip()
            if cores.lower() in comandos_saida:
                break
            sugestoes = sugerir_palavras(trie, chute, cores, historico)
            historico.append((normalizar_palavra(chute), normalizar_cores(cores)))
        except ValueError as erro:
            continue
        print("Sugestões de palavras (total de {len(sugestoes)})")
        for palavra in sugestoes:
            print(" ", palavra.upper())
            if normalizar_cores(cores) == VERDE * WORD_LENGTH:
                break
            rodada += 1
        except KeyboardInterrupt:
            print("Assistente interrompido pelo usuário.")
```

```
Informe cores com A=Amarelo, V=Verde, P=Preto. Digite 'sair' para encerrar.
Sugestões de Palavras (total de 1)
CLASS
Sugestões de Palavras (total de 1)
CLASS
Resposta correta.
```

```
In [12]: # Esta célula implementa uma versão offline do Termo para o computador jogar sozinho.
# A escolha do próximo chute usa uma heurística simples: priorizar palavras com mais letras
# distintas e depois ordenar alfabeticamente.
# Complexidade por rodada: O(H * N * L * C * L), em que H é o histórico, N é o total de
# palavras da Trie e C é a quantidade de candidatas restantes.

def escolher_proximo_chute(candidatas):
    return sorted(candidatas, key=lambda palavra: (-len(set(palavra)), palavra))[0]

def computador_joga_termo(trie, secreta, max_tentativas=6, mostrar=True):
    secreta = normalizar_palavra(secreta)
    if not trie.search(secreta):
        raise ValueError("A palavra secreta precisa existir no dicionário da Trie.")
    historico = []
    candidatas = trie.words()
    tentativas = 1
    for rodada in range(1, max_tentativas + 1):
        chute = escolher_proximo_chute(candidatas)
        cores = calcular_cores_termo(secreta, chute)
        tentativas.append((rodada, chute, cores, list(candidatas)))
        if mostrar:
            print(f"Chute #{rodada}: {chute.upper()} -> {cores} | possíveis antes do filtro: {len(candidatas)}")
            if cores == VERDE * WORD_LENGTH:
                return tentativas
            historico.append((chute, cores))
            candidatas = sugerir_palavras(trie, chute, cores, historico)
            for chute_antigo, cores_antigos in historico[:1]:
                candidatas = [p for p in candidatas if palavra_compativel(p, chute_antigo, cores_antigos)]
            if not candidatas:
                return tentativas
        return tentativas

tentativas_demo = computador_joga_termo(trie_python, "CLASS")

Chute #1: ASYNC -> AAPPA | possíveis antes do filtro: 9
Possíveis: ['ASYNC', 'AWAIT', 'BREAK', 'CLASS', 'FALSE', 'MATCH', 'RAISE', 'WHILE', 'YIELD']
Chute #2: CLASS -> VVVVV | possíveis antes do filtro: 1
Possíveis: ['CLASS']
```

```
In [13]: # Esta célula reúne demonstrações objetivas dos algoritmos implementados.
# Complexidade total das demonstrações: depende das chamadas feitas; as principais são
# O(L) para busca exata, O(P * S) para prefixo e O(N * L) para sugestão por cores.

print("Dicionário:", [p.upper() for p in PALAVRAS_RESERVADAS_PYTHON])
print("n1) Busca exata")
print("CLASS existe?", buscar_palavra(trie_python, "class"))
print("TERMO existe?", buscar_palavra(trie_python, "termo"))

print("n2) Busca por prefixo")
print("Prefixo 'ai', [p.upper() for p in buscar_por_prefixo(trie_python, "a")]
print("Prefixo 'lw', [p.upper() for p in buscar_por_prefixo(trie_python, "w")])

print("n3) Sugestões com uma resposta do Termo")
chute_demo = "await"
cores_demo = calcular_cores_termo("async", chute_demo)
print(f"Se a secreta fosse ASYNC, chute {chute_demo.upper()} retornaria {cores_demo}")
print("Sugestões:", [p.upper() for p in sugerir_palavras(trie_python, chute_demo, cores_demo)])

print("n4) Sugestões usando histórico")
historico_demo = [{"await", calcular_cores_termo("async", "await")}]
print("Novo chute BREAK contra ASYNC", calcular_cores_termo("break", "break"), calcular_cores_termo("async", "break"), historico_demo))
Dicionário: ['ASYNC', 'AWAIT', 'BREAK', 'CLASS', 'FALSE', 'MATCH', 'RAISE', 'WHILE', 'YIELD']
```

```
1) Busca exata
CLASS existe? True
TERMO existe? False

2) Busca por prefixo
Prefixo 'ai': ['ASYNC', 'AWAIT']
Prefixo 'lw': ['WHILE']

3) Sugestões com uma resposta do Termo
Se a secreta fosse ASYNC, chute AWAIT retornaria VPPPP.
Sugestões: ['ASYNC']

4) Sugestões usando histórico
Histórico: [['await', 'VPPPP']]
Novo chute BREAK contra ASYNC: PPPAP
Sugestões refinadas: [], (p.upper() for p in sugerir_palavras(trie_python, "break", calcular_cores_termo("async", "break"), historico_demo))
```

Resultados pedidos no enunciado

- A Árvore Trie é implementada a construída a partir de um dicionário formado por palavras reservadas da linguagem Python com cinco letras.
- O notebook permite informar um chute e uma sequência de cores no padrão A, V e P, sugerindo as palavras compatíveis a partir da Trie.
- As respostas anteriores podem ser acumuladas em um histórico para refinar as sugestões seguintes.
- Foram separadas células específicas para exibir a Trie e mostrar visualmente, em SVG/HTML, a execução da busca exata, da busca por prefixo e da filtragem por cores.
- As células de implementação contêm comentários com a ordem de complexidade dos algoritmos usados.
- Também foi implementada uma versão offline do Termo, na qual o computador escolhe chutes, calcula as cores e reduz iterativamente o conjunto de soluções possíveis até encontrar a palavra secreta ou atingir o limite de tentativas.